COMPLETE GUIDE

Re-ranking

The bridge between basic and production RAG

Understanding how reranking transforms RAG from a demo to a production-ready system

Reranking

Cross-Encoders

Two-Stage Retrieval

Production RAG

Qwen3

Naved Khan

Senior Gen-AI Engineer

+ Follow

What is Reranking?

DEFINITION

Reranking is an additional stage between retrieval and augmentation. It reorders retrieved documents to prioritize the most relevant ones. Think of it as precision tuning after recall-focused retrieval.

How It Works

Reranking takes the top-K documents from initial retrieval. Then it scores each document against the query using a more powerful model. Finally it reorders them by relevance score.

- **Input:** Top-K candidates from fast retrieval (usually 50-100 documents)
- **Process:** Scores each query-document pair for relevance
- **Output:** Reordered list with most relevant documents first

THINK OF IT LIKE

A librarian first finds books in the right section (fast retrieval). Then reads summaries to pick the best ones (reranking). The final selection is more precise and relevant.

Naved Khan
Senior Gen-AI Engineer

PROBLEM

The Problem Reranking Solves

THE ISSUE

Embedding-based retrieval returns topically related documents. But they're not always strictly relevant to your specific question. This is a design tradeoff, not a bug.

Real Example

Query: "How does the refund policy differ for monthly versus annual plans?"

- Embedding retrieval returns: General refund policy document
- Also returns: Pricing page mentioning annual plans
- Also returns: Billing FAQ referencing refunds
- All are semantically close, so they surface

WITH RERANKING

Reranking prioritizes chunks that explicitly compare monthly and annual refunds. Generic documents move down. The most relevant context moves up. Nothing about the data changed. What changed was how relevance was evaluated.


Naved Khan
Senior Gen-AI Engineer

+ Follow

ARCHITECTURE

Two-Stage Pipeline

THE PIPELINE

Production RAG uses a two-stage retrieval process. Stage 1 focuses on speed and recall. Stage 2 focuses on precision and relevance.

Stage 1: Fast Retrieval

Uses bi-encoder (embedding) models. Converts query and documents to vectors separately. Compares using cosine similarity. Fast and scalable. Gets top-K candidates quickly.

Stage 2: Reranking

Uses cross-encoder models. Processes query and document together. Evaluates relevance directly. Slower but more accurate. Reorders candidates by relevance.

WHY THIS WORKS

Bi-encoders are fast enough to search millions of documents. Cross-encoders are accurate enough to pick the best from hundreds. Together they give you both speed and precision.

Naved Khan
Senior Gen-AI Engineer

COMPARISON

Bi-Encoder vs Cross-Encoder

BI-ENCODER (STAGE 1)

Separately encodes query and documents into vectors. Uses cosine similarity for comparison. Document embeddings can be precomputed. Very fast retrieval over large corpora.

CROSS-ENCODER (STAGE 2)

Concatenates query and document together. Feeds combined input into transformer. Model outputs relevance score directly. More accurate but slower per pair.

Trade-offs

- **Speed:** Bi-encoder excels at scale (millions of docs). Cross-encoder works on small sets (dozens to hundreds)
- **Accuracy:** Bi-encoder good for semantic matching. Cross-encoder captures fine-grained interactions
- **Cost:** Bi-encoder cheap after precomputation. Cross-encoder requires full transformer passes per pair

Naved Khan

Senior Gen-AI Engineer

Qwen3 Reranker Family

STATE-OF-THE-ART MODELS

Qwen3 rerankers are open-source models that outperform earlier rerankers. They support up to 32K tokens context length. Available in three sizes for different use cases.

Model Options

- **Qwen3-Reranker-8B:** Maximum accuracy. Scores 77.45 on CMTEB-R, 81.22 on MTEB-Code. Best for high-stakes applications
- **Qwen3-Reranker-4B:** Balanced performance and cost. Competitive with 8B in many metrics. Great default choice
- **Qwen3-Reranker-0.6B:** Low latency, resource-efficient. Good for cost-sensitive or real-time systems

INDUSTRY USE

Companies like Fireworks AI host these models for production use. They're used in RAG systems, semantic search, and knowledge bases. The 4B model often provides the best balance.

Naved Khan
Senior Gen-AI Engineer

CODE

Implementation Example

Using Reranking API

Here's how to use reranking with Fireworks AI API. Simple interface for reranking documents based on a query.

PYTHON

```python
import requests

url = "https://api.fireworks.ai/inference/v1/rerank"

payload = {
    "model": "fireworks/qwen3-reranker-8b",
    "query": "What is RAG?",
    "documents": [# Your docs],
    "top_n": 3
}

response = requests.post(url, json=payload)
```

HOW IT WORKS

The API takes your query and list of documents. It scores each document for relevance. Returns top_n documents sorted by relevance score. Relevance scores help you understand confidence.

Naved Khan

Senior Gen-AI Engineer

USE CASES

When Reranking Matters Most

HIGH-VALUE SCENARIOS

Reranking provides the biggest impact when accuracy matters more than speed. When wrong answers have serious consequences. When queries are complex or ambiguous.

Best Use Cases

- **Complex queries:** Multi-hop reasoning, comparisons, nuanced questions
- **Domain-specific knowledge:** Legal, medical, financial systems where precision is critical
- **High-stakes applications:** Customer support, knowledge bases, research assistants
- **Ambiguous queries:** When user questions are vague or need clarification

REAL-WORLD EXAMPLE

A legal research system needs precise document retrieval. A medical diagnosis assistant requires accurate context. A financial advisor system must find relevant regulations. Reranking ensures the right information reaches the LLM.


Naved Khan
Senior Gen-AI Engineer

+ Follow

Production Best Practices

Optimization Strategies

- **Limit candidate pool:** Only rerank top 50-100 from fast retrieval. Don't rerank the full corpus
- **Balance costs:** Reranking cost should be offset by LLM savings. Fewer chunks mean lower generation costs
- **Cache results:** Cache popular queries and reranked results. Prevents recomputing for common questions
- **Monitor metrics:** Track NDCG@k, MRR, Precision@k. Use these to optimize your pipeline

COST EQUATION

RAG Savings = Cost(LLM with base chunks) - (Cost(reranker) + Cost(LLM with fewer chunks)). Choose reranking pool size to maximize this savings.

PRODUCTION TIP

Start with reranking top 50 candidates. Measure improvement in answer quality. Adjust based on your latency budget and accuracy requirements. Fine-tune thresholds over time.

Naved Khan
Senior Gen-AI Engineer

PRODUCTION

Production Considerations

Key Decisions

- **Latency vs accuracy:** Larger models are more accurate but slower. Choose based on your SLA requirements
- **Model selection:** Use 8B for maximum accuracy. Use 4B for balanced performance. Use 0.6B for low latency
- **Fine-tuning:** Fine-tune on domain-specific query-document pairs. Can improve precision significantly
- **Monitoring:** Track both accuracy metrics and operational metrics. Monitor for drift over time

EVALUATION STRATEGY

Use A/B testing to compare rerankers. Collect user feedback and click data. Continuously evaluate on test benchmarks. Compare performance with and without reranking.

VERSION CONTROL

Version your rerankers, retrieval models, and knowledge base. If a new reranker degrades performance, you need ability to roll back. Keep track of what works.

Naved Khan
Senior Gen-AI Engineer

+ Follow

Key Takeaway

THE BOTTOM LINE

Reranking is what differentiates basic RAG from production RAG. It's not optional. It's essential for reliable systems. It improves precision, reduces noise, and leads to better generation.

Why It Matters

- **Better relevance:** More accurate document selection improves answer quality
- **Reduced hallucinations:** Better context means fewer made-up facts
- **Efficient context:** Fewer but better documents save tokens and costs
- **Production-ready:** Turns a demo into something you can trust

REMEMBER

Reranking should be thought of as part of retrieval itself, not an optional add-on. In practice, it's the layer that turns a RAG pipeline into something you can trust in production.

Naved Khan
Senior Gen-AI Engineer

+ Follow

Want more content like this?

Naved Khan

Senior Gen-AI Engineer

Follow for more AI insights and RAG best practices

Like Comment Share Save